Project Planning and Estimation

Assignment 6 | Automated Data Visualization Dashboard System

The project is a dashboard that reads spreadsheet files, cleans and analyses the data, and automatically creates summaries and charts. This report estimates the software size, development effort and time required by a four-member team.

Planning basis: SRS version 0.2; first four sprints. Report date: 10 October 2026. These figures are estimates, not completed-project results. Optional AI features and later enhancements are outside the estimate.

1. Lines of Code (LOC) Estimation

LOC measures software size using source-code lines, excluding blank lines and comments. The bottom-up method estimates each module separately, then adds the results. Three values are used: optimistic (O), most likely (M) and pessimistic (P).

Expected LOC = (O + 4M + P) / 6

Module	O	M	P	Expected LOC
Application routes	100	180	300	187
File validation and reading	200	350	550	358
Data cleaning and profiling	300	500	800	517
Chart-selection rules	200	350	550	358
Chart rendering	150	300	450	300
Configuration	30	50	80	52
Page templates	150	250	400	258
CSS and JavaScript	100	180	300	187
Total	**1,230**	**2,160**	**3,430**	**2,217**

Example: application routes = (100 + 4 × 180 + 300) / 6 ≈ **187 LOC**. Adding the rounded module estimates gives **2,217 LOC**, or **2.217 KLOC**. Test code is estimated separately at 1,550 LOC and is excluded from the production size.

Cross-check using function points

The function-point estimate in Section 2 is approximately 91 FP. The original plan uses a blended conversion of 44.4 LOC per FP, based on an 80% Python/JavaScript and 20% templates/CSS split. JavaScript is used as a proxy because the cited QSM table has no Python entry.

Method	Estimated size
Bottom-up module estimate	2,217 LOC
FP conversion: low rate (91 × 27.6)	2,512 LOC
FP conversion: average rate (91 × 44.4)	4,040 LOC

Selected estimate: 2.2 KLOC. The module estimate reflects the actual design and the use of libraries such as pandas and Plotly. The higher FP-based figure is retained as a planning comparison, rather than treated as an exact prediction.

2. Function Point Estimation

Function points measure the functionality delivered to users, independently of the programming language. The IFPUG approach groups functions into five types and assigns low, average or high complexity weights.

Component	Low	Average	High
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

Functions counted in this project

The table retains the original component counts, with implementation details removed. Separate chart types count as separate outputs because they use different selection and calculation rules.

Type	Function / data group	Complexity	Points
Input	Upload and process a file	High	6
Input	Switch to another sheet	Average	4
Output	Summary cards	Average	5
Output	Warnings panel	Average	5
Output	Column information table	Low	4
Output	Line chart (R1)	Average	5
Output	Bar chart (R2)	Average	5
Output	Pie chart (R3)	Low	4
Output	Histogram (R4)	Low	4
Output	Scatter plot (R5)	Average	5
Output	Correlation heatmap (R6)	Average	5
Query	Data preview (first 10 rows)	Average	4
Query	Sheet list in the selector	Low	3
Query	Download a chart as PNG	Low	3
Internal file	Cleaned dataset	Low	7
Internal file	Column profile	Low	7
Internal file	Chart specification list	Low	7
Internal file	Error log	Low	7
External file	Uploaded spreadsheet	Average	7

An input sends data into the system; an output presents calculated information; a query retrieves existing information. Internal files are data maintained by the system, while external interface files are data maintained elsewhere.

2. Function Points: Calculation

Unadjusted function points

Component	Low	Average	High	UFP
EI	0 × 3	1 × 4	1 × 6	10
EO	3 × 4	6 × 5	0 × 7	42
EQ	2 × 3	1 × 4	0 × 6	10
ILF	4 × 7	0 × 10	0 × 15	28
EIF	0 × 5	1 × 7	0 × 10	7
Total				**97**

Adjustment for system characteristics

The plan rates 14 characteristics from 0 to 5 to reflect the system's operating needs and complexity. The ratings below total 29.

Characteristic	Rating	Characteristic	Rating
Data communications	2	Online update	1
Distributed data processing	1	Complex processing	4
Performance	3	Reusability	2
Heavily used configuration	1	Installation ease	2
Transaction rate	0	Operational ease	1
Online data entry	5	Multiple sites	2
End-user efficiency	3	Facilitate change	2

Higher ratings reflect interactive data entry, data-processing logic and performance requirements. Lower ratings reflect the simple deployment: one user processes one file at a time on an ordinary computer.

Final function-point estimate

Value Adjustment Factor (VAF) = 0.65 + 0.01 × total rating

VAF = 0.65 + 0.01 × 29 = **0.94**

Adjusted FP = UFP × VAF = 97 × 0.94 = 91.18 ≈ 91 FP

Important counting assumption

The original estimate counts temporary cleaned data, column profiles and chart specifications as internal logical files. These exist only during processing. A stricter interpretation that counts only the persistent error log would reduce UFP by 21, giving approximately 71 FP. The report retains the original 91 FP estimate with this assumption stated clearly.

3. COCOMO Effort Estimation

The Constructive Cost Model (COCOMO), developed by Barry Boehm, estimates development effort and duration from software size. Basic COCOMO is used here.

Model selection

The project uses the **organic mode**: a small team, a familiar problem and relatively few hard constraints. Reading files, summarising data and producing charts do not require new hardware or complex external integrations.

Quantity	Formula
Effort (E), in person-months	E = 2.4 × KLOC^1.05
Development time (D), in months	D = 2.5 × E^0.38
Average staff	P = E / D

KLOC means thousands of source-code lines. One person-month represents 152 hours of work. Effort is the total work required; duration is the elapsed development time.

Calculation using the main estimate

Software size = 2,217 / 1,000 = **2.217 KLOC**

Effort = 2.4 × (2.217)^1.05 ≈ **5.54 person-months**

Development time = 2.5 × (5.54)^0.38 ≈ **4.79 months**

Average staff = 5.54 / 4.79 ≈ **1.16 people**

Comparison of the two size estimates

	Bottom-up (2.22 KLOC)	Top-down average (4.04 KLOC)
Effort (E)	5.5 PM	10.4 PM
Development time (D)	4.8 months	6.1 months
Average staff (P)	1.2 people	1.7 people
Productivity (LOC / E)	400 LOC per PM	389 LOC per PM
Effort in hours (× 152)	842 hours	1,580 hours
Hours per member (÷ 4)	210 hours	395 hours

What these results mean

The main estimate is approximately **842 hours in total**, or **210 hours per member** for a team of four. Over the 39-working-day schedule, this is roughly 27 hours per member per week.

COCOMO’s nominal duration is about 4.8 months. The project schedule is shorter because the plan assigns work in parallel to four members. This is a strongly compressed plan; adding people does not guarantee a proportional reduction in time.

Basic COCOMO was calibrated on older software projects and does not directly account for modern frameworks or reusable libraries. Its results should therefore be treated as a planning reference, with actual effort recorded during development.

4. Project Scheduling with PERT / CPM

PERT estimates activity durations using three possible outcomes. The Critical Path Method (CPM) then identifies the longest dependent sequence of activities, which determines the earliest project completion.

Expected time = (o + 4m + p) / 6

Here, o = optimistic, m = most likely and p = pessimistic duration. All durations are in working days. The team works Monday to Friday; public holidays and exam weeks are not excluded.

Activities, dependencies and time estimates

ID	Activity	Predecessors	o	m	p	Expected
A	Project setup	-	1	2	3	2
B	Upload page and routes	A	1	3	5	3
C	File validation and reading	A	5	7	15	8
D	Prepare test files	A	2	3	4	3
E	Data cleaning and type detection	D	6	7	14	8
F	Statistics and totals	E	2	3	4	3
G	Chart-selection rules	E	4	7	10	7
H	Chart rendering	G	3	5	7	5
I	Dashboard layout	B, F	3	6	9	6
J	Integration and sheet selector	C, H, I	2	4	6	4
K	Error handling and logging	J	2	4	6	4
L	Complete automated tests	D, J	4	6	8	6
M	Performance testing	K	2	2	8	3
N	Browser and user testing	K	1	2	3	2
P	User documentation	J	2	3	4	3
O	Final fixes and review	L, M, N, P	1	3	5	3

Worked example

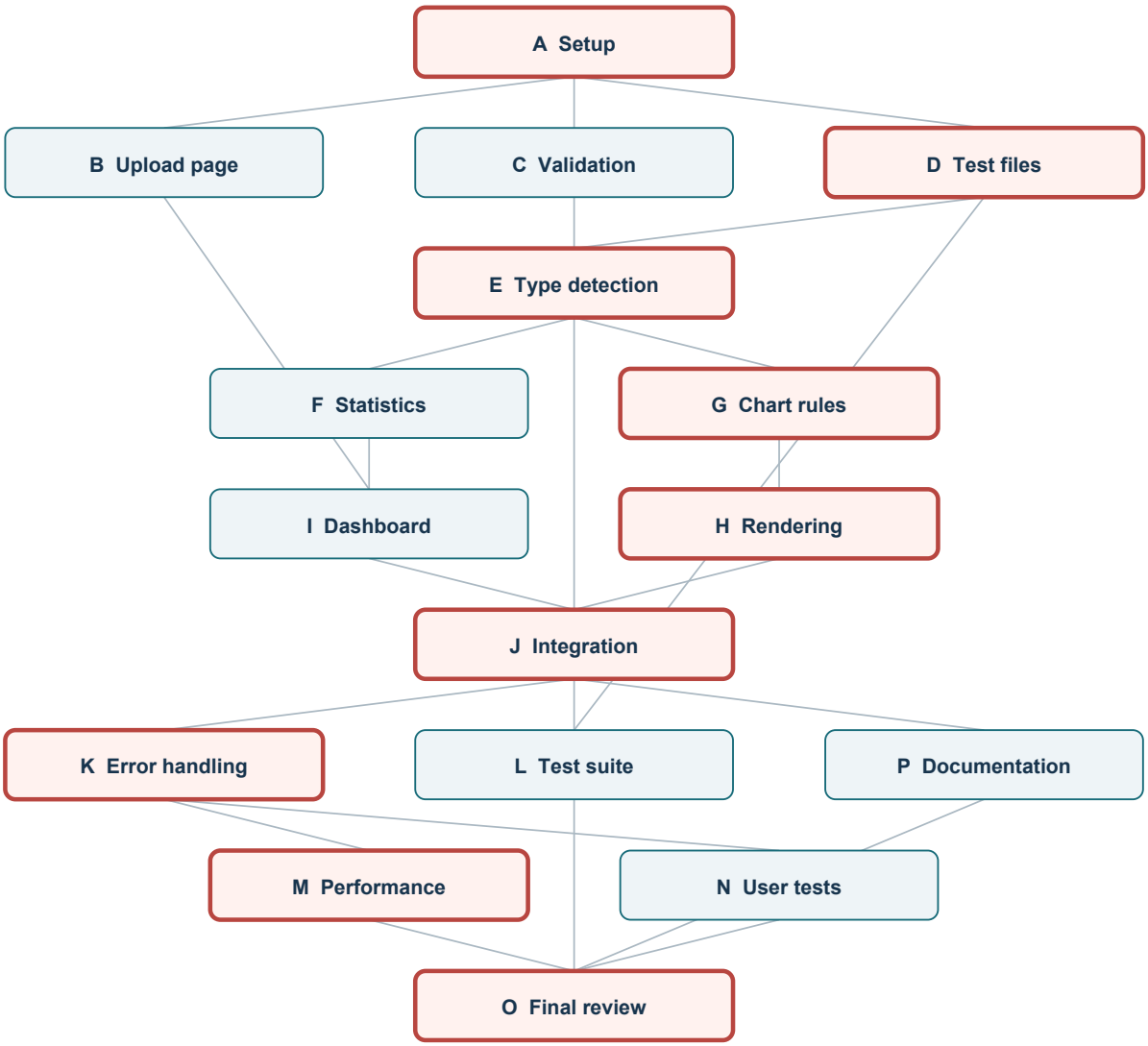
File validation and reading (C): (5 + 4 × 7 + 15) / 6 = **8 working days**. This activity can begin after project setup (A).

Why some activities can run in parallel

Data cleaning (E) can begin once test files (D) are ready, without waiting for the complete validator (C). Chart rendering (H) follows chart-selection rules (G). Integration (J) waits until validation, rendering and the dashboard layout are ready.

4. Scheduling: Dependency Diagram

Activity-on-node network. Critical activities have a thick red border. Read the dependencies from top to bottom.



Critical path: A → D → E → G → H → J → K → M → O

4. Scheduling: Critical Path and Result

A forward pass finds the earliest start (ES) and finish (EF). A backward pass finds the latest start (LS) and finish (LF) that still allow completion on time.

EF = ES + duration; LS = LF - duration; slack = LS - ES. An activity with zero slack is critical.

ID	Duration	ES	EF	LS	LF	Slack	Critical
A	2	0	2	0	2	0	Yes
B	3	2	5	16	19	14	-
C	8	2	10	17	25	15	-
D	3	2	5	2	5	0	Yes
E	8	5	13	5	13	0	Yes
F	3	13	16	16	19	3	-
G	7	13	20	13	20	0	Yes
H	5	20	25	20	25	0	Yes
I	6	16	22	19	25	3	-
J	4	25	29	25	29	0	Yes
K	4	29	33	29	33	0	Yes
L	6	29	35	30	36	1	-
M	3	33	36	33	36	0	Yes
N	2	33	35	34	36	1	-
P	3	29	32	33	36	4	-
O	3	36	39	36	39	0	Yes

Critical path

A → D → E → G → H → J → K → M → O

Duration = 2 + 3 + 8 + 7 + 5 + 4 + 4 + 3 + 3 = **39 working days**.

Schedule milestone	Date
Project start	Tuesday, 1 September 2026
Expected completion	Friday, 23 October 2026

Planning priorities

Prepare test files early. They are on the critical path, so testing preparation cannot be postponed until the end.

Protect the data-to-chart sequence. Delays in data cleaning, chart rules or rendering move the expected completion date.

Use available slack carefully. The upload page and validator have 14 and 15 working days of slack respectively, providing room to support critical activities.

The planned duration is an estimate, not a guaranteed deadline. Monitor progress and update the schedule if activity durations or dependencies change.